HERMES · PHASE 0 · LESSON 0003 · LAB EXERCISE 02-MODEL-CLIENT-SDK · ~45–60 MIN

The SDK Absorbs the Six Responsibilities

You rewrite exercise 01's raw client with the official `@anthropic-ai/sdk`, aimed at the same mock. Every responsibility you wrote by hand still exists. Find where each one went.

Why this matters for Hermes

Every model call Hermes makes uses this SDK's mechanics: authentication, retries, typed errors, cancellation. Scenario step S4 (model calls) puts that layer below Hermes policy. The six-responsibilities thread continues: the SDK absorbs all six, and the wire does not change.

1 · Nothing changes on the wire

Exercise 02 points the official SDK at exercise 01's mock through one option, `baseURL`. Start the mock, run the SDK client, and watch the server's terminal. The mock logs the same `POST /v1/messages` and answers with the same canned fixture. It cannot tell the two clients apart. An SDK does an API's chores. It does not change the API.

The circled numbers below are lesson 0001's six responsibilities: ① endpoint, ② authentication, ③ API version, ④ request and response shape, ⑤ errors, ⑥ cancellation.

```
POST /v1/messages HTTP/1.1                    ① the endpoint
host: localhost:8899                          the capture server. Your mock is :8787
accept: application/json
user-agent: Anthropic/JS 0.113.0              the SDK names itself
x-stainless-retry-count: 0
x-stainless-timeout: 600
   … six more x-stainless-* telemetry headers …
anthropic-version: 2023-06-01                 ③ sent for you, pinned by the SDK release
x-api-key: mock-key                           ② from the apiKey option
content-type: application/json
content-length: 102

{"model":"claude-opus-4-8","max_tokens":1024,  ④ the request shape
 "messages":[{"role":"user","content":"What is an SDK?"}]}
```

The body is byte-for-byte the raw client's body: the same 102 bytes, measured. The headers are a superset. The SDK adds its own name and telemetry, and the mock ignores every extra header.

	Exercise 01, you wrote	Exercise 02, it became
① endpoint	``fetch(`${BASE_URL}/v1/messages`)``	the `baseURL` option. The path lives inside `client.messages.create`.
② authentication	`"x-api-key": API_KEY`	the `apiKey` option, defaulting to `ANTHROPIC_API_KEY` from the environment
③ version	`"anthropic-version": "2023-06-01"`	nothing to write. The SDK sends it.
④ request and response shape	a hand-built body, a hand-written interface, an `as`	typed `MessageCreateParams` in, typed `Message` out. Still an assertion (§4).
⑤ errors	`if (!res.ok)` plus a hand-parsed envelope	typed error classes, and automatic retry that obeys `retry-after`
⑥ cancellation	`signal` in the fetch options	`signal` in the request options, plus a per-call `timeout`

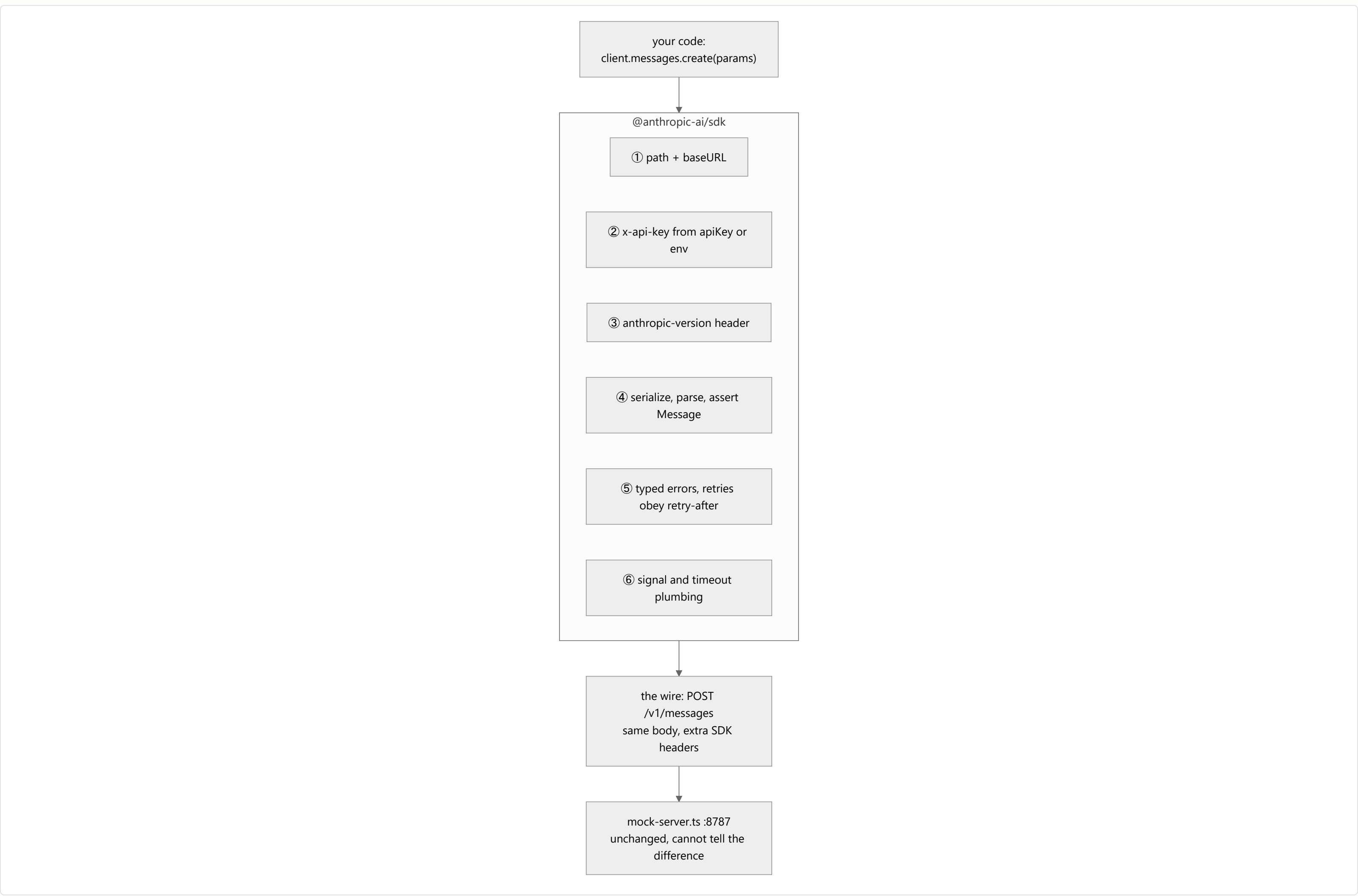
your code:
client.messages.create(params)
@anthropic-ai/sdk
① path + baseURL
② x-api-key from apiKey or env
③ anthropic-version header
④ serialize, parse, assert Message
⑤ typed errors, retries obey retry-after
⑥ signal and timeout plumbing
the wire: POST /v1/messages same body, extra SDK headers
mock-server.ts :8787 unchanged, cannot tell the difference

Diagram 1. The SDK is a layer between your code and the wire, not a change to the wire.

2 · Construction is configuration

Exercise 01 scattered the six responsibilities across one function. The SDK gathers the durable ones into an *API client*: an object that holds connection settings and exposes the API's operations as typed methods. Construction is configuration. Calls are use.

```
const client = new Anthropic({
  baseURL: "http://localhost:8787", // ① default: ANTHROPIC_BASE_URL, else api.anthropic.com
  apiKey: "mock-key",               // ② default: ANTHROPIC_API_KEY from the environment
  maxRetries: 2,                    // ⑤ the default
  timeout: 600_000,                 // ⑥ the default, in MILLISECONDS
});
```

Everything set here is a default for the client's whole life. A single call can override the transport through *request options*: a second argument of per-call transport settings, kept separate from the request body. Params say what you are asking. Options say how this one call behaves on the wire.

```
await client.messages.create(
  { model, max_tokens, messages },                 // params: the ④ request shape
  { headers: { "x-mock-scenario": "rate-limit" },  // options: this call only
    maxRetries: 0, timeout: 200, signal: controller.signal },
);
```

Read the call one link at a time.

Piece	Syntax	Meaning	What exists at runtime
`import Anthropic from "@anthropic-ai/sdk"`	default import	the SDK's entry class	a class object from `node_modules`
`new Anthropic({ … })`	constructor call	build one configured client	the client object with your settings
`client.messages`	property read	the Messages resource	an object the constructor created
`.create(params, options?)`	method call	one `POST /v1/messages`	a promise of the parsed `Message`

The declaration of `create` has three overloads, and the params type decides the return type. Without `stream: true` the call promises one `Message`. A signature tells TypeScript what may be called. The implementation does the runtime work. The SDK abstracts chores away from your application code, not away from TypeScript.

Every default above comes from the SDK's own files, not from its docs. A *declaration file* is a `.d.ts` file that states a package's types and documented defaults, with no implementation. The constructor's JSDoc in `node_modules/@anthropic-ai/sdk/client.d.ts` states the environment fallbacks, the 10-minute timeout, and `maxRetries: 2`. For a generated SDK they are the authoritative description of its surface. The exercise starts there.

3 · Errors and cancellation, measured

Responsibility ⑤ used to be your `if (!res.ok)` branch. The SDK raises a *typed error* instead: a failure raised as a distinct class per failure family, carrying structured fields. The hierarchy lives in `core/error.d.ts`.

```
// core/error.d.ts (abridged)            // what your catch receives
export declare class APIError {           RateLimitError {
  readonly status: number | undefined;      status: 429,
  readonly type: ErrorType | null;          type: "rate_limit_error",
  readonly requestID: string | null;        requestID: "req_mock_0001",
  readonly headers: Headers | undefined;    headers: { "retry-after": "5", … },
}                                          }
export declare class RateLimitError
  extends APIError<429, Headers> {}
```

A 429 does not reach your catch at once, because the SDK retries first. *Retry with exponential backoff* means re-sending a failed request after waiting longer between attempts. The SDK retries connection errors, 408, 409, 429 and 5xx answers, twice by default, and it obeys the `retry-after` header. Measured against the mock's scripted 429: three requests in the server log, 10.1 seconds of client silence, then one `RateLimitError`. With `maxRetries: 0` the same failure surfaces in 17 milliseconds.

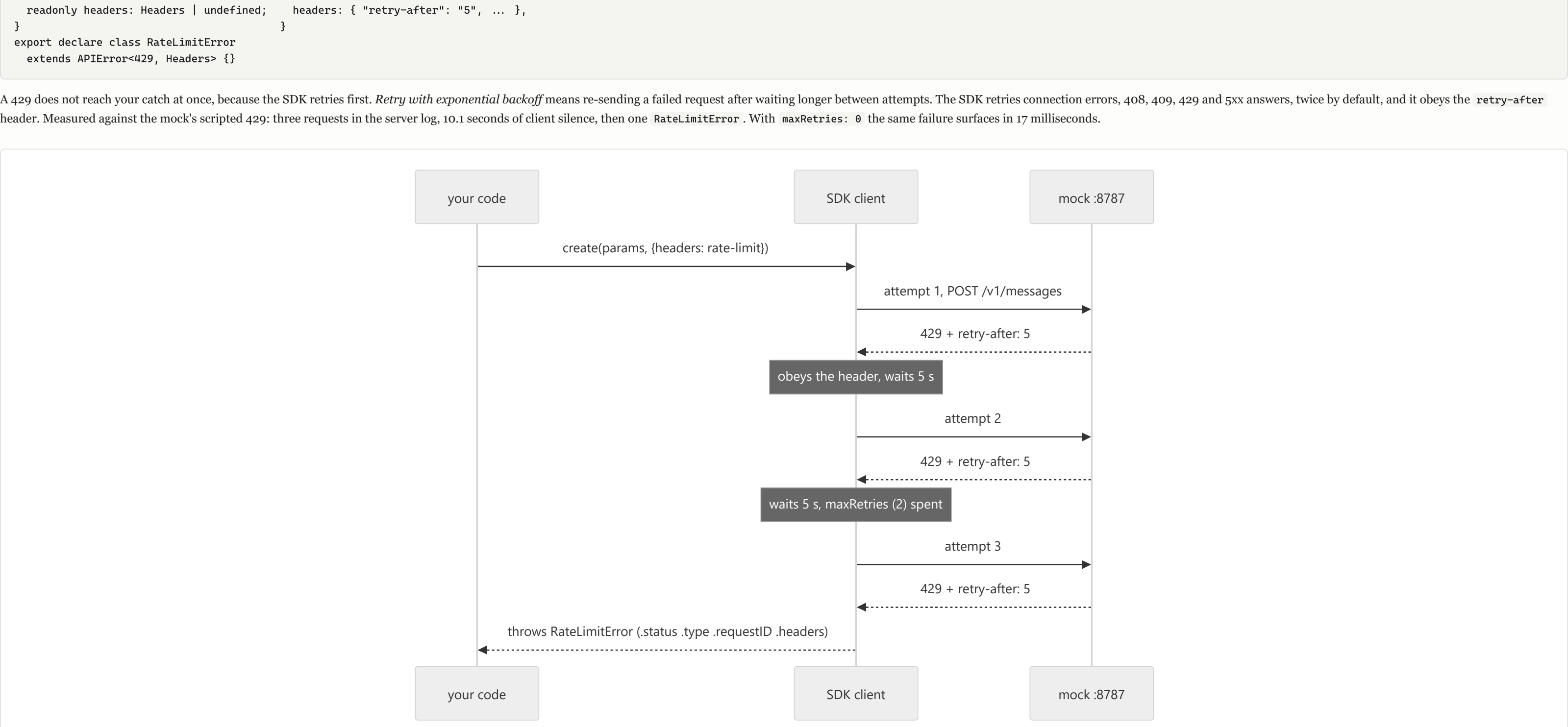
your code
SDK client
mock :8787
create(params, {headers: rate-limit})
attempt 1, POST /v1/messages
429 + retry-after: 5
obeys the header, waits 5 s
attempt 2
429 + retry-after: 5
waits 5 s, maxRetries (2) spent
attempt 3
429 + retry-after: 5
throws RateLimitError (.status .type .requestID .headers)
your code
SDK client
mock :8787

Diagram 2. The default retry policy against the mock's scripted 429.

Responsibility ⑥ has two knobs. A `timeout` of 200 milliseconds expires inside the mock's 400-millisecond latency. The SDK treats a timeout like a connection error: three attempts, then `APIConnectionTimeoutError` after about 2.1 seconds.

Cancellation is stopping an in-flight request on purpose. Pass an `AbortController`'s `signal` in the request options and call `abort()`. One attempt stops, `APIUserAbortError` surfaces at once, and the SDK retries nothing.

4 · What moved to compile time, and what did not

In exercise 01 you misspelled `messages` and learned about it from a 400. The same typo now never reaches a running process.

```
error TS2769: Object literal may only specify known properties, but 'mesages'
does not exist in type 'MessageCreateParamsNonStreaming'.
Did you mean to write 'messages'?
```

Remove `max_tokens` and the check fails again with a missing-property error. Two of the mock's 400 gates moved into `tsc`. Three things deliberately did not move.

The model id still compiles when misspelled. The declaration reads `Model = 'claude-opus-4-8' | … | (string & {})`, a union of known ids plus one member that admits any string. New models must keep working against old SDKs, so the catalog stays on the server.

The key's value still passes everywhere. A value like `apiKey: "definitely-not-real"` satisfies the compiler and the mock alike. Lesson 0002's line between presence and authenticity has not moved.

The response is still trusted, not checked. The SDK attaches its `Message` type to parsed bytes by assertion, the same pipeline lesson 0001 §3 dissected, now generated instead of hand-written. Nothing at runtime checks that the bytes match the type. Lesson 0005 closes that gap: Hermes validates model-shaped data with Zod at the boundary.

New today.

Five ideas are new.

- An API client holds connection settings once. Every call inherits them.
- Params and options are separate arguments: what you ask, and how one call behaves.
- The SDK retries some failures for you and obeys `retry-after`.
- Failures arrive as typed error classes with structured fields.
- Declaration files state a generated SDK's types and defaults. Read them first.

Reprise: the six responsibilities, the base URL, presence against authenticity, and the parse-then-assert pipeline.

5 · The exercise

The commands and the full experiment table live in `hermes-sdk-lab/02-model-client-sdk/README.md`. The exercise has five steps.

1. Read the declarations first. Open `node_modules/@anthropic-ai/sdk/client.d.ts`, find the constructor, and read the documented defaults for `apiKey`, `baseURL`, `timeout` and `maxRetries`.
2. Break it before you build it. Write the create call with `mesages`, run `pnpm typecheck`, and read the error. Fix it, delete `max_tokens`, read again, undo.
3. Send it. Terminal 1 runs `pnpm mock`, terminal 2 runs `pnpm request`. On the 200, print the same four things as exercise 01 and diff the two clients' output.
4. Run the ⑤ and ⑥ experiments: the rate-limit header with and without `maxRetries: 0`, the 200-millisecond timeout, the 100-millisecond abort. Watch both terminals. Count the requests the mock logs.
5. Close the loop. For each failure, name the layer that caught it: the compiler, the SDK, or the mock. Then say what only the real API could have caught.

6 · Exercise: who catches it now?

Classify each failure

Answer from memory before you click. Feedback is immediate.

You write `mesages` instead of `messages` in the params.

The compiler catches it
The SDK catches it
Only a server can catch it

The model id is misspelled: `claude-opsu-4-8`.

The compiler catches it
The SDK catches it
Only a server can catch it

The params object has no `max_tokens`.

The compiler catches it
The SDK catches it
Only a server can catch it

The server answers 429 with a `retry-after` header.

The compiler catches it
The SDK catches it
Only a server can catch it

The `apiKey` value is not a genuine key.

The compiler catches it
The SDK catches it
Only a server can catch it

The mock's ~400 ms latency exceeds your 200 ms `timeout`.

The compiler catches it
The SDK catches it
Only a server can catch it

Score: 0 / 0

7 · Quiz

Four questions

Q1. You swap exercise 01's raw `fetch` for the SDK. What changes in the request the mock receives?

The body gains an SDK envelope around your params
The transport switches to a binary wire format
Nothing but extra SDK headers the mock ignores
The calls are batched so fewer requests are sent

Q2. The mock answers 429 plus `retry-after: 5` every time. With default settings, what does your process observe?

One request is sent, then the error surfaces instantly
Three requests are sent, an error after ten seconds
Two requests are sent, then an error after a second
Six requests are sent, an error after thirty seconds

Q3. At runtime, what does the SDK's `Message` response type guarantee about the bytes received?

Nothing, since the type is asserted over parsed JSON
Each field is checked while the bytes are parsed
Only required fields are verified, never optional
The shape is validated once per client instance

Q4. `model: "claude-opsu-4-8"` compiles without complaint. Why did the SDK's authors allow that?

Model names are checked by the runtime, not the compiler
The field is typed as a plain string for compile speed
A generator bug left the union open to every string
New models must keep working against older SDK types

Score: 0 / 0

8 · Say it in your own words

After the lab, answer these four in the chat, from memory.

1. Define SDK in one sentence, without the words "tools" or "platform". What does it do to the API's chores, and what changes on the wire?
2. Your raw client could only print `retry-after`. What does the SDK do with it? Name the two knobs that control that behavior, one on the client, one per call.
3. Compare the SDK's `Message` type with the `MessageResponse` interface you wrote by hand. What differs at runtime, and what is the real difference?
4. Lesson 0006's gateway wraps this SDK. Name two things it owns itself, because the SDK gives mechanics and no policy.

What this lesson names, and where it is built

Capability	Status	Where
a gateway that wraps this SDK under Hermes policy	implemented	lesson 0006, lab exercise 05
runtime validation of the response bytes	implemented	lesson 0005, lab exercise 04
streaming delivery and mid-stream cancellation	implemented	lesson 0004, lab exercise 03

If you keep only four sentences, keep these. An SDK does an API's chores and leaves the wire unchanged. The client object holds connection settings, and request options override them for one call. The SDK retries some failures and obeys `retry-after`, so one call can become three requests. The response type is still an assertion, so runtime validation stays your job.

Primary source

Read next: the anthropic-sdk-typescript README sections on retries, timeouts, errors and cancellation. Check each claim against what you observed. Then open `node_modules/@anthropic-ai/sdk/core/error.d.ts` and read the whole error hierarchy. The wire rules underneath both stay the Messages API reference.

Verified against `@anthropic-ai/sdk` 0.113.0, Node 22.14.0 and pnpm 11.17.0, as resolved in `pnpm-lock.yaml`. The wire block in §1 is a real SDK request, captured 2026-08-18 against a local logging server on a private port. The retry, timeout and abort numbers were measured against the mock on 2026-07-24. The mock approximates the real API's error wording. This lesson's terms are registered under `wiki/terms/`.

Next: lesson 0004 sets `stream: true` on this same call and follows the response as it becomes a process.